Simple CTF

TryHackMe — Easy

Campo	Valor
Plataforma	TryHackMe
Dificuldade	Easy
Autor	Gustavo Vieira de Araujo
Data de Resolução	10 de abril de 2026
CVE	CVE-2019-9053
Tipo de Vulnerabilidade	SQL Injection (time-based blind)

1. Visão Geral

Simple CTF é uma máquina Linux de nível Easy da plataforma TryHackMe (https://tryhackme.com). Meu ponto de partida foi apenas o IP do alvo, visto que não foi fornecida nenhuma outra informação. O caminho percorrido foi: mapeamento dos serviços expostos com Nmap, descoberta de uma aplicação web rodando o CMS Made Simple versão 2.2.8 em um subdiretório identificado por enumeração, busca por vulnerabilidades públicas nessa versão do CMS, execução de um exploit de SQL Injection (CVE-2019-9053) que extraiu o hash da senha do administrador do banco de dados, quebra do hash com Hashcat e acesso à máquina via SSH. A escalada de privilégios foi possível por uma configuração incorreta do sudo que permitia executar o vim como root sem senha.

2. Reconhecimento

2.1 Varredura de Portas — Nmap

O primeiro passo foi descobrir quais serviços estavam rodando na máquina. Para isso, usei o Nmap (https://nmap.org/), uma ferramenta padrão de varredura de redes. O comando executado foi:

```
nmap -sC -sV -p- --min-rate 5000 -oN nmap_result.txt <TARGET_IP>
```

Cada flag teve um motivo específico, pesquisado antes ou durante a execução:

- **-sC (Script Scan):** Ao consultar a documentação oficial do Nmap, aprendi que essa flag ativa o Nmap Scripting Engine (NSE) com os scripts da categoria default. Esses scripts executam verificações automáticas, como detectar se o FTP aceita login anônimo, identificar cabeçalhos HTTP e enumerar certificados SSL. Sem essa flag, teria que fazer essas verificações manualmente depois.
- **-sV (Version Detection):** Pelo manual do Nmap, essa flag faz a ferramenta enviar probes adicionais para identificar a versão exata de cada serviço em vez de apenas detectar que a porta está aberta. Isso é fundamental para a fase de exploração: saber que é Apache 2.4.18 permite buscar vulnerabilidades específicas dessa versão.
- **-p- (All Ports):** Por padrão, o Nmap varre apenas as 1.000 portas mais comuns, selecionadas pelo arquivo interno nmap-services. Pesquisei sobre esse comportamento e entendi que em máquinas CTF, serviços propositalmente aparecem em portas fora desse intervalo. O -p- força a varredura de todas as 65.535 portas TCP possíveis. A decisão foi acertada: o SSH nesta máquina estava na porta 2222.
- **--min-rate 5000:** Varrer 65.535 portas com detecção de versão seria muito lento sem ajuste de velocidade. Encontrei na documentação do Nmap que essa flag define o número mínimo de pacotes por segundo enviados. Com 5.000 pacotes/s, a varredura completa foi concluída em cerca de 1 minuto.
- **-oN nmap_result.txt:** A documentação descreve três formatos de saída: -oN (normal, legível), -oX (XML) e -oG (grepável). Escolhi -oN por ser o mais fácil de ler e consultar depois. Salvar o output é importante porque os resultados do Nmap servem de referência durante toda a resolução.

Saída relevante obtida:

```
PORT     STATE SERVICE VERSION
21/tcp   open  ftp     vsftpd 3.0.3
| ftp-anon: Anonymous FTP login allowed (FTP code 230)
|_Can't get directory listing: TIMEOUT
80/tcp   open  http    Apache httpd 2.4.18 ((Ubuntu))
| http-robots.txt: 2 disallowed entries
|_/ /openemr-5_0_1_3
|_http-title: Apache2 Ubuntu Default Page: It works
2222/tcp open  ssh     OpenSSH 7.2p2 Ubuntu 4ubuntu2.8
```

O resultado mostrou três portas abertas. A linha "ftp-anon: Anonymous FTP login allowed" foi gerada pelo script ativado com -sC. Já o SSH na porta 2222 confirmou a importância do “-p-” : essa porta não estaria em uma varredura padrão.

Porta	Serviço	Versão	Observação
21/tcp	FTP	vsftpd 3.0.3	Login anônimo habilitado (detectado pelo script ftp-anon do -sC)
80/tcp	HTTP	Apache 2.4.18	Script do Nmap detectou robots.txt com entradas
2222/tcp	SSH	OpenSSH 7.2p2	Porta não padrão — só apareceu por causa do -p-

2.2 Enumeração do FTP

O script ftp-anon indicou que a porta 21 aceita login anônimo. Pesquisei o que isso significa do ponto de vista de segurança e encontrei que o acesso anônimo ao FTP é uma configuração historicamente problemática: qualquer pessoa pode se conectar usando o usuário "anonymous" e qualquer string como senha, sem precisar de credenciais reais. Isso frequentemente expõe arquivos internos como configurações, backups ou credenciais deixados descuidadamente no servidor.

Tentei conectar para verificar o conteúdo disponível:

```
ftp <TARGET_IP>
Name: anonymous
Password: [enter]
230 Login successful.
ftp> ls
(timeout — sem listagem de diretório)
```

O login funcionou, mas o servidor retornou timeout ao tentar listar o diretório. Pesquisei esse comportamento e aprendi que o FTP opera em dois modos: ativo e passivo. No modo ativo, o servidor tenta abrir uma conexão de dados de volta para o cliente, o que frequentemente é bloqueado por firewalls. Como o FTP não expôs nenhum arquivo acessível, descartei esse vetor e passei para o HTTP.

2.3 Enumeração HTTP — robots.txt e Feroxbuster

O Nmap sinalizou que o servidor HTTP possui um arquivo robots.txt. Pesquisei o propósito desse arquivo e descobri que ele é um padrão definido pelo Robots Exclusion Protocol e sua função é instruir rastreadores de mecanismos de busca a não indexar determinadas páginas. Do ponto de vista de segurança, administradores frequentemente listam nele diretórios sensíveis que não querem que apareçam no Google, o que acidentalmente os revela para qualquer pessoa que consulte o arquivo.

```
curl http://<TARGET_IP>/robots.txt
User-agent: *
Disallow: /
Disallow: /openemr-5_0_1_3
```

O path /openemr-5_0_1_3 não respondeu ao acesso direto. Para mapear os diretórios existentes, usei o Feroxbuster, uma ferramenta de enumeração de diretórios por força bruta. Pesquisei as diferenças entre as ferramentas disponíveis, como por exemplo o Gobuster, Dirb, Dirsearch e Feroxbuster e optei pelo Feroxbuster por dois motivos encontrados na documentação oficial: é recursivo por padrão (ao encontrar um diretório, entra nele e continua a enumeração automaticamente) e tem performance superior às alternativas mais antigas. O Gobuster, por exemplo, não tem recursividade nativa na versão que eu tinha instalada.

```
feroxbuster -u http://<TARGET_IP> -w raft-medium-directories.txt -t 50 -d 2 -x
php,txt,bak,zip -C 404
```

Detalhamento de cada flag:

- **-u http://<TARGET_IP>:** Define a URL base. O Feroxbuster concatena cada palavra da wordlist a essa URL para testar sua existência.
- **-w raft-medium-directories.txt:** A wordlist raft-medium-directories.txt faz parte do projeto SecLists e contém ~30.000 nomes de diretórios comuns coletados de aplicações web reais. Pesquisei quais wordlists são mais recomendadas para enumeração web em CTFs e encontrei referências frequentes a essa lista por equilibrar cobertura e tempo de execução.
- **-t 50:** Define 50 threads paralelas. O padrão já é 50, mas entender o parâmetro me ajudou a saber que poderia reduzir esse número caso o servidor começasse a retornar erros por sobrecarga.
- **-d 2:** Limita a recursão a dois níveis de profundidade. Sem esse limite, o Feroxbuster continuaria entrando indefinidamente em subdiretórios, resultando em varreduras longas e desnecessárias.
- **-x php,txt,bak,zip:** Além de testar nomes de diretórios, testa as mesmas palavras com essas extensões. Pesquisei quais extensões são mais relevantes em aplicações PHP: .bak e .txt são comuns em backups acidentais, .php testa scripts diretamente e .zip pode revelar pacotes de instalação ou backup de código-fonte.
- **-C 404:** Filtra da saída todas as respostas com código HTTP 404 (Not Found). Sem esse filtro, a saída seria inundada com milhares de linhas para cada palavra que não existe no servidor.

Saída relevante:

```
200  GET  http://<TARGET_IP>/robots.txt
301  GET  http://<TARGET_IP>/simple         =>  http://<TARGET_IP>/simple/
301  GET  http://<TARGET_IP>/simple/admin   =>  http://<TARGET_IP>/simple/admin/
301  GET  http://<TARGET_IP>/simple/uploads =>  http://<TARGET_IP>/simple/uploads/
200  GET  http://<TARGET_IP>/simple/config.php
200  GET  http://<TARGET_IP>/simple/index.php
```

O diretório /simple/ se destacou. A presença simultânea de config.php, uploads/, admin/ e index.php é característica da estrutura de um CMS.

2.4 Identificação do CMS e Versão

Pesquisando sobre como identificar CMSs em aplicações web, aprendi que muitos inserem sua identidade e versão em metatags HTML ou no rodapé. Usei o curl combinado com grep para filtrar apenas as linhas com palavras como "generator", "version" ou "powered", que são os campos mais comuns:

```
curl http://<TARGET_IP>/simple/ | grep -i "generator\|version\|powered"
<meta name="Generator" content="CMS Made Simple - Copyright (C) 2004-2019." />
[...]
This site is powered by CMS Made Simple version 2.2.8
```

Confirmado: CMS Made Simple versão 2.2.8. Com a versão exata identificada, passei para a busca por vulnerabilidades conhecidas.

3. Exploração

3.1 Busca por Exploits com Searchsploit

Antes de tentar qualquer análise manual, pesquisei se já existiam exploits públicos para o CMS Made Simple 2.2.8. Usei o Searchsploit, uma ferramenta que permite consultar o banco de dados do Exploit-DB localmente. A vantagem é poder combinar com filtros de terminal sem depender de conexão com a internet. O Exploit-DB é o maior repositório público de exploits, mantido pela Offensive Security.

```
searchsploit "cms made simple 2.2"
---------------------------------------------- ----------------------------------
 Exploit Title                                |  Path
---------------------------------------------- ----------------------------------
CMS Made Simple 2.2.5  - (Authenticated) RCE  | php/webapps/44976.py
CMS Made Simple 2.2.7  - (Authenticated) RCE  | php/webapps/45793.py
CMS Made Simple < 2.2.10 - SQL Injection      | php/webapps/46635.py
CMS Made Simple 2.2.14 - Arbitrary File Upload | php/webapps/48779.py
---------------------------------------------- ----------------------------------
```

Analisei cada resultado: os exploits de RCE tinham "(Authenticated)" no título — só funcionam após login na aplicação, e eu ainda não tinha credenciais. O 46635.py ("CMS Made Simple < 2.2.10 - SQL Injection") era o único sem requisito de autenticação e cobria a versão 2.2.8. Pesquisei a CVE associada e encontrei a CVE-2019-9053, que descreve uma SQL Injection no parâmetro m1_idlist do módulo de busca. O tipo é time-based blind: o script infere cada caractere medindo se o servidor demora mais do que o esperado para responder, usando a função SLEEP() do banco de dados.

3.2 Execução do Exploit — CVE-2019-9053

Executei o exploit passando apenas a URL base do CMS — o script já conhece os parâmetros vulneráveis:

```
python3 46635.py -u http://<TARGET_IP>/simple/
```

O exploit levou vários minutos, testando cada caractere das credenciais um a um via injeções com sleep. Saída final relevante:

```
[+] Salt for password found: 1dac0d92e9fa6bb2
[+] Username found: mitch
[+] Email found: admin@admin.com
[+] Password found: 0c01f4468bd75d7a84c7eb73846e8d96
```

Pesquisei o que é um salt no contexto de armazenamento de senhas e aprendi que é um valor aleatório adicionado à senha antes de aplicar a função de hash, alem disso seu objetivo é impedir que duas senhas iguais gerem o mesmo hash e dificultar o uso de rainbow tables. O salt é fundamental para a próxima etapa: sem ele, não seria possível configurar o Hashcat corretamente.

Campo extraído	Valor	Para que serve
Username	mitch	Usuário para autenticação SSH
Salt	1dac0d92e9fa6bb2	Necessário para recriar o hash no Hashcat com o modo correto
Hash	0c01f4468bd75d7a84c7eb73846e8d96	Senha em md5(salt+pass); alvo da quebra por dicionário

3.3 Quebra de Hash com Hashcat

Com o hash e o salt em mãos, o próximo passo era descobrir a senha original. Pesquisei como funciona a quebra de hashes e aprendi que o processo é o inverso do que o sistema faz ao armazenar a senha: pego uma lista de senhas candidatas, gero o hash de cada uma e comparo com o hash alvo quando coincidem, a senha foi encontrada. O ponto crítico era identificar o modo de hash correto. Pesquisei o formato de armazenamento de senhas do CMS Made Simple e encontrei referências indicando que ele usa md5(salt + password). Consultei a página de modos do Hashcat e identifiquei que esse formato corresponde ao modo 20 (md5($salt.$pass)). Usar o modo 0 (MD5 puro) produziria hashes incompatíveis e nunca encontraria a correspondência, mesmo que a senha estivesse na wordlist.

```
echo -n '<HASH>:<SALT>' > hash.txt
hashcat -m 20 hash.txt rockyou.txt
```

Detalhamento das flags:

- **-m 20:** Define o algoritmo de hash como md5($salt.$pass). Obtido diretamente na documentação do Hashcat (seção Hash modes). Sem esse parâmetro correto, a execução inteira seria em vão.
- **hash.txt:** O arquivo com o hash no formato hash:salt, exatamente como o Hashcat espera para o modo 20. Usei echo -n para evitar que uma quebra de linha fosse adicionada ao final, o que corromperia o hash.
- **rockyou.txt:** Uma das wordlists mais famosas em segurança, originada de um vazamento de dados real de 2009. Contém mais de 14 milhões de senhas reais em ordem de frequência de uso. Pesquisei quais wordlists são mais usadas em CTFs e o rockyou.txt é a recomendação padrão em praticamente todos os recursos de referência para esse tipo de ataque.

Resultado obtido:

```
0c01f4468bd75d7a84c7eb73846e8d96:1dac0d92e9fa6bb2:secret
```

O Hashcat encontrou a correspondência rapidamente. Com isso, tenho as credenciais completas: usuário mitch e senha “secret”.

3.4 Acesso Inicial via SSH

Com as credenciais em mãos e o SSH identificado na porta 2222, tentei fazer login. Usei a flag -p 2222 para especificar a porta não padrão, visto que sem ela, o cliente SSH tentaria a porta 22 e falharia sem nenhuma mensagem de erro útil:

```
ssh -p 2222 mitch@<TARGET_IP>
```

Login bem-sucedido. A flag de usuário estava no diretório home de mitch:

```
mitch@Machine:~$ cat ~/user.txt
G00d j0b, keep up!
```

Listei o conteúdo de /home para mapear outros usuários — etapa que faço para entender o ambiente antes de partir para a escalada de privilégios:

```
mitch@Machine:~$ ls /home/
mitch  sunbath
```

Há um segundo usuário, sunbath, mas seu diretório não estava acessível com as permissões de mitch. Passei para a escalada de privilégios.

4. Escalada de Privilégios

4.1 Enumeração de Permissões sudo

O primeiro comando que executei após obter acesso inicial foi sudo -l. Pesquisei sobre técnicas de escalada de privilégios em Linux e encontrei que verificar permissões sudo é sempre o ponto de partida recomendado, uma vez que e simples, rápido e não gera nenhum ruído adicional no sistema. Ferramentas de enumeração automática como o LinPEAS cobrem isso e muito mais, mas optei por começar manualmente para entender o que estava vendo antes de recorrer a scripts:

```
mitch@Machine:~$ sudo -l
User mitch may run the following commands on Machine:
    (root) NOPASSWD: /usr/bin/vim
```

Pesquisei o que NOPASSWD significa na configuração do sudo e aprendi que é uma diretiva do arquivo /etc/sudoers que dispensa a autenticação por senha para aquele comando específico. Pesquisei sobre os riscos de conceder sudo a editores de texto e descobri que o vim possui capacidade nativa de executar comandos do sistema operacional e código Python a partir de dentro do editor o que transforma essa permissão em execução arbitrária de código como root.

4.2 Abuso do Vim para Shell Root

Para encontrar a técnica exata, consultei o GTFOBins (gtfobins.github.io), um repositório colaborativo que documenta como binários Unix comuns podem ser abusados quando executados com permissões elevadas. Procurei pelo vim na seção sudo e encontrei duas abordagens:

- :!bash ou :!sh — o sinal ! no modo de comando do vim executa um comando externo do shell.
- :py3 import os; os.execl(...) — executa código Python pelo interpretador embutido no vim.

Pesquisei a diferença entre as duas. O :!bash cria um subprocesso filho do vim. Já o os.execl substitui completamente o processo atual pelo novo processo (/bin/sh), herdando o mesmo PID e os privilégios de root do vim. O reset antes do exec sh é necessário para limpar o estado do terminal, que fica corrompido quando o vim assume o controle do TTY:

```
mitch@Machine:~$ sudo vim -c ':py3 import os; os.execl("/bin/sh", "sh", "-c", "reset; exec
sh")'
```

A flag -c do vim executa um comando interno imediatamente ao abrir o editor, sem precisar interagir com a interface. O vim foi iniciado como root e o Python substituiu seu processo por uma shell /bin/sh com os mesmos privilégios. Com a shell root ativa, li a flag final:

```
# cat /root/root.txt
W3ll d0n3. You made it!
```

5. Conclusão

A cadeia de exploração da máquina Simple CTF partiu de uma varredura completa com Nmap que revelou FTP anônimo (sem conteúdo acessível), um servidor HTTP com CMS Made Simple 2.2.8 descoberto via Feroxbuster, e SSH na porta não padrão 2222. A versão desatualizada do CMS era vulnerável à CVE-2019-9053, explorada pelo script 46635.py que extraiu, via SQL Injection time-based blind, o hash MD5 com salt das credenciais do administrador. O Hashcat no modo 20 com a wordlist rockyou.txt recuperou a senha em texto claro. Com as credenciais, o acesso inicial foi obtido via SSH. A escalada de privilégios veio de uma permissão sudo NOPASSWD mal configurada para o vim, explorada via Python embutido para substituir o processo do editor por uma shell root e capturar a flag final.

6. Referências

- **Documentação oficial do Nmap — flags, NSE e formatos de saída:**
  https://nmap.org/book/man.html
- **Nmap Scripting Engine (NSE) — scripts da categoria default:**
  https://nmap.org/nsedoc/categories/default.html
- **CVE-2019-9053 — National Vulnerability Database (NVD):**
  https://nvd.nist.gov/vuln/detail/CVE-2019-9053
- **Exploit 46635.py — Exploit-DB, CMS Made Simple < 2.2.10 SQL Injection:**
  https://www.exploit-db.com/exploits/46635
- **OWASP — Blind SQL Injection:**
  https://owasp.org/www-community/attacks/Blind_SQL_Injection
- **Hashcat — Hash modes (lista completa de modos e exemplos):**
  https://hashcat.net/wiki/doku.php?id=hashcat
- **GTFOBins — vim (sudo):**
  https://gtfobins.github.io/gtfobins/vim/
- **Feroxbuster — documentação e comparação com outras ferramentas:**
  https://github.com/epi052/feroxbuster
- **SecLists — wordlists utilizadas (raft-medium-directories.txt, rockyou.txt):**
  https://github.com/danielmiessler/SecLists
- **Robots Exclusion Protocol — especificação do robots.txt:**
  https://www.robotstxt.org/robotstxt.html